

SECURITY_GUIDE

HelixTrack Security Guide — 2026-06-21

Overview

Security architecture and best practices for HelixTrack.

Authentication

JWT Authentication

- **Algorithm:** HS256
- **Expiration:** 24 hours
- **Refresh:** Optional refresh tokens

Password Security

- **Hashing:** bcrypt
- **Minimum length:** 8 characters
- **Requirements:** Uppercase, lowercase, number, special character

OAuth (Planned)

- **Providers:** Google, GitHub, Microsoft
 - **Flow:** Authorization code grant
-

Authorization

Role-Based Access Control (RBAC)

- **Roles:** Admin, Manager, Developer, Viewer
- **Permissions:** Create, Read, Update, Delete

- **Resources:** Projects, Tickets, Users, Settings

Permission Matrix

Role	Projects	Tickets	Users	Settings
Admin	CRUD	CRUD	CRUD	CRUD
Manager	CRUD	CRUD	Read	Read
Developer	Read	CRUD	Read	Read
Viewer	Read	Read	Read	Read

Data Security

Encryption

- **At rest:** SQLite encryption (SQLCipher)
- **In transit:** TLS 1.3
- **Backups:** Encrypted backups

Data Isolation

- **Multi-tenant:** Space-based isolation
- **Database per space:** Separate SQLite files
- **File storage:** Isolated per space

API Security

Input Validation

- **SQL Injection:** Parameterized queries
- **XSS:** Output encoding
- **CSRF:** Token validation

Rate Limiting

- **Login attempts:** 5 per minute
- **API requests:** 100 per minute per user
- **File uploads:** 10MB max

CORS

- **Origins:** Configurable
- **Methods:** GET, POST, PUT, DELETE
- **Headers:** Authorization, Content-Type

Audit Logging

Logged Events

- User login/logout
- CRUD operations
- Permission changes
- Security events

Log Format

```
{  
  "id": "uuid",  
  "timestamp": "ISO-8601",  
  "username": "string",  
  "resource": "string",  
  "resource_id": "string",  
  "action": "string",  
  "allowed": true/false,  
  "reason": "string",  
  "ip_address": "string",  
  "user_agent": "string"  
}
```

Security Best Practices

Development

- Use HTTPS in production
- Validate all inputs
- Sanitize outputs
- Use parameterized queries
- Implement CSRF protection
- Set secure headers

Deployment

- Use environment variables for secrets
- Rotate secrets regularly
- Monitor audit logs
- Backup encrypted data
- Update dependencies

Monitoring

- Monitor failed login attempts

- Track API usage
 - Alert on suspicious activity
 - Review audit logs
-

Vulnerability Management

Regular Tasks

- Update dependencies
- Run security scans
- Review audit logs
- Test penetration

Security Scans

- **SAST:** Static analysis
 - **DAST:** Dynamic analysis
 - **Dependency:** npm audit, go mod verify
-

Incident Response

Detection

- Monitor audit logs
- Alert on anomalies
- Track failed logins

Response

1. Identify scope
 2. Contain breach
 3. Investigate root cause
 4. Implement fix
 5. Notify affected users
-

Cross-references

- [Architecture](#)
- [Deployment Guide](#)

Sources verified 2026-06-22: /Volumes/T7/Projects/helix_track/auth/ (sibling HelixTrack source tree) , <https://github.com/Helix-Track/Everything>

REPO-STATE-DERIVED (per §11.4.99 the sources are the cross-referenced HelixTrack auth source, following the docs/ARCHITECTURE.md precedent). Cross-referenced on 2026-06-22: - **JWT/Bearer auth CONFIRMED** in the present sibling tree helix_track/auth/{ARCHITECTURE,README}.md; the umbrella github.com/Helix-Track/Everything confirms a dedicated Auth service in the HelixTrack architecture. - **Negative finding:** generic security best-practices in this guide (TLS, incident response, rate-limiting) are HelixTrack-project-internal guidance and were NOT re-verified against any external security-standard's latest revision in this pass — only the auth-mechanism (JWT) was confirmed against the source tree. Re-verify any cited external standard/CVE/library guidance per §11.4.99 before treating it as current.